FDR Portal: Phase-by-Phase Build Guide

This guide explains how to build each part of the Security Deposit / FDR Management Portal, in order. For every phase it gives the design decisions, schemas, endpoints, logic, screens and checks. It deliberately leaves out scheduling, team split and presentation material. Use it alongside the Build Roadmap, which holds the scope tiers (Must, Should, Could).

Conventions used throughout

- **Stack assumed:** React + TypeScript (Vite), Node + TypeScript (Express), PostgreSQL 16, Docker Compose. Everything translates directly to FastAPI or NestJS.
- **Dates and money:** deadline dates (issue, maturity, due) are `DATE`, never timestamps. Event times are `TIMESTAMPTZ`. Money is `NUMERIC(14,2)`, never floats. Compute the business date in Asia/Kolkata.
- **One clock:** no business logic calls `new Date()` or `NOW()`. Everything reads the app clock built in Phase 1.
- **Transactions:** every state change runs inside a database transaction that also writes an audit event.
- **API:** prefix `/api/v1`, JSON, errors shaped as `{ code, message, details }`. IDs are UUIDs.
- **Layering:** routes call services, services call repositories. Business rules live only in services.
- **Verification versus lifecycle:** the lifecycle status says where an instrument is in its life. A separate `verification_state` says how much of it is proven. The Roadmap listed Verified as a lifecycle step; here it is tracked by `verification_state`, and `ACTIVE` means an officer has accepted the instrument.

Phase 0: Environment and skeleton

Goal: one command starts the whole system and every piece can reach the others.

0.1 Repository layout

```
fdr-portal/
  docker-compose.yml
  .env.example
  backend/
    src/
      routes/ services/ repositories/ middleware/ domain/ jobs/ db/
      app.ts server.ts
    migrations/
    tests/
  frontend/
    src/ pages/ components/ api/ auth/ hooks/
  mock-bank/
    src/
  seed/
  docs/
```

0.2 Docker Compose

Four services: `db` (postgres:16), `backend` (port 3000), `frontend` (port 5173), `mock-bank` (port 4000). The backend depends on `db` and reads the mock bank address from an environment variable.

Environment variables in `.env.example`: `DATABASE_URL`, `JWT_SECRET`, `MOCK_BANK_URL`, `STORAGE_DIR`, `TZ=Asia/Kolkata`.

0.3 Backend skeleton

- `app.ts` wires the JSON body parser, a request-id middleware, routes and the error handler.
- **Middleware:** `authenticate` (verifies the token), `requireRole(...roles)`, `validate(schema)` (zod), and `errorHandler`, which maps domain errors (NotFound, Forbidden, Conflict, ValidationFailed, InvalidTransition) to HTTP codes (404, 403, 409, 422, 409).
- **Database access:** a `pg` pool and a `withTransaction(fn)` helper that opens a transaction, passes the client to `fn`, then commits or rolls back.
- **Migrations:** numbered SQL files run in order (`001_init.sql`, `002_...`). Run them automatically on startup in development.

0.4 Frontend skeleton

Vite, React Router, TanStack Query, Tailwind. An `api` module wraps `fetch`: it attaches the token, parses `{ code, message, details }` errors and throws typed errors. A layout component shows a sidebar whose entries depend on the user role.

0.5 Check

`GET /api/v1/health` returns database status. The frontend home page displays it. `mock-bank` answers `GET /health`.

Phase 1: Foundation

Goal: data model, authentication, roles, the clock, the state machine and the audit hook. Every later phase builds on these.

1.1 Core schema (migration 001)

```
CREATE TYPE user_role AS ENUM
('institution','officer','approver','admin','auditor');
CREATE TYPE instrument_type AS ENUM ('FDR','SECURITY_DEPOSIT');
CREATE TYPE instrument_status AS ENUM (
  'DRAFT','SUBMITTED','UNDER_SCRUTINY','RETURNED','ACTIVE','EXPIRING',
  'MATURED','RENEWED','LAPSED','RELEASE_REQUESTED','RELEASED');
CREATE TYPE trust_tier AS ENUM ('CLAIMED','DOCUMENT','BANK_CONFIRMED');

CREATE TABLE institutions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aicte_id text UNIQUE NOT NULL,
  name text NOT NULL,
  region text NOT NULL,
  contact_email text
);

CREATE TABLE users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text UNIQUE NOT NULL,
  password_hash text NOT NULL,
  role user_role NOT NULL,
  institution_id uuid REFERENCES institutions(id),
```

```
  created_at timestamptz NOT NULL DEFAULT now(),
  CHECK ((role = 'institution') = (institution_id IS NOT NULL))
);

CREATE TABLE instruments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  institution_id uuid NOT NULL REFERENCES institutions(id),
  type instrument_type NOT NULL,
  reference_no text NOT NULL,
  issuing_bank text NOT NULL,
  branch text,
  amount numeric(14,2) NOT NULL CHECK (amount > 0),
  issue_date date NOT NULL,
  maturity_date date NOT NULL,
  beneficiary_name text NOT NULL,
  status instrument_status NOT NULL DEFAULT 'DRAFT',
  status_before_release instrument_status,
  verification_state text NOT NULL DEFAULT 'UNVERIFIED',
  lien_marked boolean NOT NULL DEFAULT false,
  assigned_officer_id uuid REFERENCES users(id),
  parent_instrument_id uuid REFERENCES instruments(id),
  submitted_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (issuing_bank, reference_no),
  CHECK (maturity_date > issue_date)
);

CREATE TABLE field_provenance (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  instrument_id uuid NOT NULL REFERENCES instruments(id) ON DELETE CASCADE,
  field_name text NOT NULL,
  value text NOT NULL,
  tier trust_tier NOT NULL DEFAULT 'CLAIMED',
  source text,
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (instrument_id, field_name)
);

CREATE TABLE documents (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  instrument_id uuid NOT NULL REFERENCES instruments(id) ON DELETE CASCADE,
  doc_type text NOT NULL,
```

```
  filename text NOT NULL,
  storage_path text NOT NULL,
  mime_type text NOT NULL,
  size_bytes integer NOT NULL,
  sha256 text NOT NULL,
  uploaded_by uuid NOT NULL REFERENCES users(id),
  uploaded_at timestamptz NOT NULL
);

CREATE TABLE events (
  id bigserial PRIMARY KEY,
  actor_id uuid,
  actor_label text NOT NULL,
  action text NOT NULL,
  entity_type text NOT NULL,
  entity_id uuid NOT NULL,
  before_json jsonb,
  after_json jsonb,
  created_at timestamptz NOT NULL,
  prev_hash text,
  hash text
);

CREATE TABLE app_clock (
  id int PRIMARY KEY CHECK (id = 1),
  mode text NOT NULL DEFAULT 'REAL',
  simulated_now timestamptz
);
INSERT INTO app_clock (id) VALUES (1);
```

Add indexes on `instruments(institution_id)`, `instruments(status)`, `instruments(maturity_date)`, `events(entity_type, entity_id)` and `documents(instrument_id)`.

1.2 The app clock

```
// domain/clock.ts
export async function now(db): Promise<Date> {
  const c = await db.one('SELECT mode, simulated_now FROM app_clock WHERE id
= 1');
  return c.mode === 'SIMULATED' ? c.simulated_now : new Date();
}
export async function today(db): Promise<string> {            // 'YYYY-MM-DD'
in IST
  const d = await now(db);
  return new Intl.DateTimeFormat('en-CA', { timeZone: 'Asia/Kolkata'
}).format(d);
}
export async function setSimulated(db, date: Date) { /* mode = SIMULATED */ }
export async function advanceDays(db, n: number) { /* simulated_now += n days
*/ }
export async function resetToReal(db) { /* mode = REAL */ }
```

Expose admin-only endpoints `GET /clock`, `POST /clock/advance {days}`, `POST /clock/set {date}`, `POST /clock/reset`. Every call writes an audit event, because changing time changes what the system does. In Phase 4 the advance endpoint also triggers the evaluation job.

1.3 Audit hook

One function, used by every mutation. Phase 8 changes only its body.

```
export async function recordEvent(tx, e: {
  actor: { id?: string; label: string };
  action: string;                 // e.g. INSTRUMENT_SUBMITTED
  entityType: string; entityId: string;
  before?: unknown; after?: unknown;
}) {
  const at = await now(tx);
  await tx.none(
    'INSERT INTO events (actor_id, actor_label, action, entity_type,
entity_id, before_json, after_json, created_at) VALUES
($1,$2,$3,$4,$5,$6,$7,$8)',
    [e.actor.id ?? null, e.actor.label, e.action, e.entityType, e.entityId,
e.before ?? null, e.after ?? null, at]);
}
```

Automatic actions use `actor.label = 'system'`. Use verb-style action names consistently, since reports and the passport timeline are built from them.

1.4 State machine

Keep the allowed transitions in one table and enforce them in one function.

```
const ALLOWED: Record<Status, Status[]> = {
  DRAFT: ['SUBMITTED'],
  SUBMITTED: ['UNDER_SCRUTINY', 'RETURNED'],
  UNDER_SCRUTINY: ['ACTIVE', 'RETURNED'],
  RETURNED: ['SUBMITTED'],
  ACTIVE: ['EXPIRING', 'RELEASE_REQUESTED'],
  EXPIRING: ['MATURED', 'RENEWED', 'RELEASE_REQUESTED'],
  MATURED: ['RENEWED', 'LAPSED', 'RELEASE_REQUESTED'],
  LAPSED: ['RENEWED', 'RELEASE_REQUESTED'],
  RELEASE_REQUESTED: ['RELEASED', 'ACTIVE', 'EXPIRING', 'MATURED', 'LAPSED'],
  RENEWED: [],
  RELEASED: [],
};

export async function transition(tx, inst, to: Status, actor, extra?) {
  if (!ALLOWED[inst.status].includes(to)) throw new
InvalidTransition(inst.status, to);
  const before = { status: inst.status };
  await tx.none('UPDATE instruments SET status = $1, updated_at = now() WHERE
id = $2', [to, inst.id]);
  await recordEvent(tx, { actor, action: 'STATUS_' + to, entityType:
'instrument', entityId: inst.id, before, after: { status: to, ...extra } });
}
```

`RELEASE_REQUESTED` can return to the earlier status if the request is rejected. Store that earlier status in `status_before_release` when entering `RELEASE_REQUESTED`, and restore it on rejection.

Never update `status` anywhere except through `transition`.

1.5 Authentication and access

- `POST /auth/login {email, password}` returns `{ token, user }`. Compare with bcrypt. The token (8 hours) carries `sub`, `role`, `institutionId`.

- `authenticate` loads the user from the token on each request. `requireRole('officer','approver')` rejects others with 403.
- **Institution scoping helper:** `assertCanSee(user, instrument)`. If the user has the `institution` role and `instrument.institution_id` is not theirs, respond 404 (not 403), so existence is not leaked.
- Login rate limit: 5 attempts per minute per IP and email.
- Seed one user per role plus two institution users (different institutions) for access tests.

1.6 Frontend foundation

- `AuthContext` holds token and user, persists to `sessionStorage`, and redirects to login on 401.
- `ProtectedRoute roles={[...]}` guards pages.
- Role homes (empty for now): institution home, officer queue, approver home, admin home, auditor home. The sidebar shows only what the role may use.

1.7 Checks

- Login succeeds and fails correctly.
- An institution user cannot fetch another institution's instrument (404).
- `transition` rejects an illegal move and records an event for a legal one.
- Advancing the clock changes `today()`.

Phase 2: Institution submission and documents

Goal: an institution can create an instrument, attach documents, edit it, and submit it for scrutiny.

2.1 Rules table for required documents (migration 002)

```
CREATE TABLE rules (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  type text NOT NULL,            -- 'required_docs' | 'reminder' |
'escalation' | 'risk_weights' | ...
  config jsonb NOT NULL,
  active boolean NOT NULL DEFAULT true
);
INSERT INTO rules (name, type, config) VALUES
 ('Required documents', 'required_docs',
  '{\"FDR\": [\"FDR_COPY\", \"BANK_LETTER\"], \"SECURITY_DEPOSIT\":
[\"DEPOSIT_CONFIRMATION\"]}');
```

The same table later holds reminder, escalation and risk rules, so the admin screen in Phase 8 can edit all of them.

2.2 Endpoints

Method and path	Who	Purpose
`POST /instruments`	institution	Create a draft
`PATCH /instruments/:id`	institution	Edit while `DRAFT` or `RETURNED`
`POST /instruments/:id/documents`	institution	Upload one document (multipart)
`DELETE /instruments/:id/documents/:docId`	institution	Remove while `DRAFT` or `RETURNED`
`GET /instruments/:id/documents/:docId/download`	institution (own), officer, approver, auditor	Authorised download
`POST /instruments/:id/submit`	institution	Submit for scrutiny
`GET /instruments`	all roles (scoped)	List with filters and paging

Method and path	Who	Purpose
`GET /instruments/:id`	all roles (scoped)	Detail

2.3 Validation

Use the same zod schema on the client and server (share it from a `shared/` package or duplicate it carefully).

- `reference_no`: 6 to 30 characters, letters, digits and hyphens.
- `issuing_bank`: chosen from a list (the same list the mock bank knows).
- `amount`: greater than 0 and below a sanity ceiling (for example 100 crore), two decimals.
- `issue_date`: not after `today()`.
- `maturity_date`: after `issue_date`, and not before `today()` at submission.
- `beneficiary_name`: required, trimmed, internal spaces collapsed.
- `type`: one of the enum values.

Business rules run in the service at submit time and return `422` with `details` listing every failing field, so the form can show all errors at once.

2.4 Document upload

- Use `multer` with disk storage, a 10 MB limit and a single-file field.
- Allow PDF, PNG and JPEG only. Check the real type from the file's first bytes (for example with the `file-type` package), not from the extension or the browser's mime header.
- Compute SHA-256 while saving.
- Store at `STORAGE_DIR/<instrumentId>/<uuid>.<ext>`. Never build the path from the original filename. Keep the original name only in the database, sanitised for display.
- If the same checksum already exists on the same instrument, accept it but return a `duplicateOf` hint so the UI can warn.
- **Download endpoint:** check authorisation first, then stream the file with `Content-Disposition: attachment`, and write a `DOCUMENT_ACCESSED` event. Files are never served by a public static route.

2.5 Submit logic

1. Load the instrument in a transaction with `FOR UPDATE`.
2. Check ownership, then that status is `DRAFT` or `RETURNED`.
3. Run field validation and compute missing documents from the `required_docs` rule. If anything is missing, return `422` listing exactly what.
4. Upsert one `field_provenance` row for each of: `reference_no`, `issuing_bank`, `amount`, `issue_date`, `maturity_date`, `beneficiary_name`, with tier `CLAIMED` and source `institution`.
5. Set `submitted_at` from the app clock and `verification_state = 'UNVERIFIED'`.
6. Call `transition(... 'SUBMITTED')`. This writes the audit event.

A second submit on an already submitted instrument fails with an invalid-transition error, which makes the endpoint safe against double clicks.

2.6 Editing rules

- Only the owning institution may edit, and only in `DRAFT` or `RETURNED`.
- Every edit records before and after values as an event.
- On resubmission after a return, any field whose value changed goes back to tier `CLAIMED`, and `verification_state` returns to `UNVERIFIED`. This rule matters: changing a value must never keep a confirmation that belonged to the old value.

2.7 Frontend

- A three-step form: **Details**, **Documents**, **Review and submit**. Save as draft at every step.
- Show server errors next to the exact fields using `details`.
- Document panel: pick the document type, upload, list with size and checksum prefix, remove button while editable.
- Institution home: a table of instruments with a status chip, maturity date, days remaining (computed on the server from the app clock) and a clear call to action when an instrument is `RETURNED`.

2.8 Edge cases to handle

- Upload after submission is rejected.
- Wrong file type with a renamed extension is rejected.

- Duplicate bank and reference number returns `409` with a readable message.
- A user who edits in two tabs gets a clear conflict, not silent overwrite (compare `updated_at` sent by the client).

2.9 Checks

- Submit with a missing document fails and names the document.
- A full submission creates six provenance rows, one `SUBMITTED` event and a status change.
- A different institution cannot download the file.
- Each download creates an access event.

Phase 3: Scrutiny, mock bank and trust tiers

Goal: officers review instruments, request bank verification, and see claimed against confirmed values.

3.1 The mock bank service

A separate Node process with its own data, exposing:

- `POST /verify` with `{ reference_no, bank, amount, issue_date, maturity_date, beneficiary_name }`.
- `GET /records` and `POST /reload` (development only, for showing and resetting the bank's data).
- `GET /health`.

Its data is a JSON file of bank records `{ bank, reference_no, amount, issue_date, maturity_date, beneficiary_name, status }` produced by the seed script. Behaviour:

1. If the bank is in a configured **offline** list, return HTTP 503 with `{ result: 'UNAVAILABLE' }`.
2. Look up `(bank, reference_no)`. If absent, return `{ result: 'NOT_FOUND' }`.
3. If the bank is in the configured **slow** list, the first call for a given reference returns `{ result: 'PENDING', retry_after_seconds: 10 }` and remembers the time. Calls after that interval return the real result.
4. Otherwise compare each submitted field with the record and return:

```
{
  'result': 'MISMATCH',
  'bank': 'Example Bank',
  'response_id': 'rsp_8f3a...',
  'responded_at': '2026-10-26T10:15:00Z',
  'fields': {
    'amount': { 'submitted': '2500000.00', 'confirmed': '2000000.00',
'match': false },
    'maturity_date': { 'submitted': '2027-03-31', 'confirmed': '2027-03-31',
'match': true }
  }
}
```

(`result` is `CONFIRMED` when every field matches.) The bank record is the source of truth, so a tampered instrument in the seed data is simply one whose bank record differs from what the institution claims.

3.2 Schema (migration 003)

```
CREATE TABLE verifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  instrument_id uuid NOT NULL REFERENCES instruments(id),
  source text NOT NULL,
  request_payload jsonb NOT NULL,
  response_payload jsonb,
  result text NOT NULL,        -- CONFIRMED | MISMATCH | NOT_FOUND | PENDING
| UNAVAILABLE
  requested_by uuid NOT NULL REFERENCES users(id),
  requested_at timestamptz NOT NULL
);

CREATE TABLE discrepancies (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  instrument_id uuid NOT NULL REFERENCES instruments(id),
  field_name text NOT NULL,
  claimed_value text NOT NULL,
  confirmed_value text,
  status text NOT NULL DEFAULT 'OPEN',   -- OPEN | RESOLVED
  resolution_note text,
  resolved_by uuid REFERENCES users(id),
  created_at timestamptz NOT NULL,
  resolved_at timestamptz
```

```
);
CREATE UNIQUE INDEX one_open_discrepancy_per_field
  ON discrepancies (instrument_id, field_name) WHERE status = 'OPEN';

CREATE TABLE observations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  instrument_id uuid NOT NULL REFERENCES instruments(id),
  author_id uuid NOT NULL REFERENCES users(id),
  text text NOT NULL,
  severity text NOT NULL DEFAULT 'INFO',  -- INFO | CONCERN | BLOCKING
  created_at timestamptz NOT NULL
);
```

3.3 Verification service

Call the bank **outside** any database transaction, so a slow network never holds locks.

```
async function requestVerification(instrumentId, officer) {
  const inst = await repo.getInstrument(instrumentId);          // read
  requireStatus(inst, ['UNDER_SCRUTINY']);
  const payload = buildPayload(inst);                            // normalised
values
  const res = await callBank(payload, { timeoutMs: 5000 });    // network, no
tx
  await withTransaction(async tx => {
    await insertVerification(tx, inst, payload, res, officer);
    await applyResult(tx, inst, res, officer);
  });
}
```

`applyResult` by outcome:

- **CONFIRMED:** set tier `BANK_CONFIRMED` (source `bank:<response_id>`) on all six fields; `verification_state = 'CONFIRMED'`; auto-resolve any open discrepancies with the note `Confirmed by bank response <id>`.
- **MISMATCH:** for each field with `match = false`, insert an `OPEN` discrepancy (skip if one is already open), leaving that field at its current tier. Fields that match become `BANK_CONFIRMED`. `verification_state = 'MISMATCH'`.
- **NOT_FOUND:** treat as a mismatch on `reference_no` with the confirmed value empty and the reason `No record at bank`.

- **PENDING:** `verification_state = 'PENDING'`. The officer re-requests later (see the retry hint).
- **UNAVAILABLE or timeout:** `verification_state = 'UNAVAILABLE'`. The case needs manual confirmation.

Every outcome writes an event (`VERIFICATION_REQUESTED`, `VERIFICATION_RESULT`) and stores the full request and response.

3.4 Comparison and normalisation

Normalise both sides before comparing, so cosmetic differences do not create false discrepancies.

- Amounts: parse to two decimals, compare as strings of that form.
- Dates: ISO `YYYY-MM-DD`.
- Names: uppercase, trim, collapse spaces, remove dots and commas.
- Reference numbers: uppercase, strip spaces.

Do the comparison in the mock bank (it decides `match`) and again defensively in the main app when building the discrepancy card values.

3.5 Trust tier rules

- **CLAIMED:** only the institution provided the value.
- **DOCUMENT:** an officer confirmed the value against an attached document. Endpoint `POST /instruments/:id/fields/:field/check-document {documentId}`. It may raise `CLAIMED` to `DOCUMENT`, and never lowers `BANK_CONFIRMED`.
- **BANK_CONFIRMED:** a bank response matched the value.
- Editing a field (Phase 2 rule) always resets it to `CLAIMED`.
- The tier shown for the instrument header is the lowest tier among the six fields, because an instrument is only as proven as its weakest field.

3.6 Officer endpoints

Method and path	Purpose
`GET /scrutiny/queue`	Filterable queue of submitted instruments
`POST /instruments/:id/claim`	Officer takes the case: `SUBMITTED` to `UNDER_SCRUTINY`, sets `assigned_officer_id`

Method and path	Purpose
`POST /instruments/:id/verify`	Request bank verification
`POST /instruments/:id/observations`	Record an observation
`POST /instruments/:id/return`	Return to institution; reason is mandatory
`POST /instruments/:id/accept`	Move to `ACTIVE` (rules below)
`POST /discrepancies/:id/resolve`	Resolve with a mandatory note

Accept rules. Accept is allowed only when all required documents are present and either `verification_state = 'CONFIRMED'`, or the officer supplies an explicit override note (for banks with no verification route). An override is stored as an event and shown on the passport as *Accepted without bank confirmation*. Any open `BLOCKING` observation or open discrepancy makes accept return `409` with the exact reasons.

3.7 Queue query

```
SELECT i.id, i.reference_no, i.issuing_bank, i.amount, i.maturity_date,
       i.verification_state, i.status, inst.name AS institution, inst.region,
       (SELECT count(*) FROM discrepancies d WHERE d.instrument_id = i.id AND
d.status = 'OPEN') AS open_discrepancies,
       i.submitted_at
FROM instruments i JOIN institutions inst ON inst.id = i.institution_id
WHERE i.status IN ('SUBMITTED','UNDER_SCRUTINY')
  AND ($1::text IS NULL OR inst.region = $1)
  AND ($2::text IS NULL OR i.issuing_bank = $2)
  AND ($3::text IS NULL OR i.verification_state = $3)
ORDER BY i.submitted_at ASC
LIMIT $4 OFFSET $5;
```

3.8 Completeness check

A pure function `findGaps(instrument, documents, rules)` returns a list such as `Missing document: BANK_LETTER`, `Field not bank-confirmed: amount`, `Open discrepancy: beneficiary_name`. Use it in the review screen (as a checklist), in the accept guard, and later in reports.

3.9 Frontend

- **Queue page:** table with filters, ageing column and a discrepancy count.
- **Review page:** left, the six fields with a trust badge each; right, tabs for Documents (with preview and a *checked against this document* action), Verification history and Observations.
- **TrustBadge component:** three states with an icon and a text label (never colour alone): Claimed, Document-checked, Bank-confirmed.
- **DiscrepancyCard component:** two columns, *Institution claimed* and *Bank confirmed*, differing field highlighted, buttons to resolve with a note or to return the case to the institution.
- Action bar: Verify, Add observation, Return, Accept. Disable Accept and show the reasons from the guard.

3.10 Checks

- A clean instrument turns every field bank-confirmed and shows `CONFIRMED`.
- A tampered amount produces exactly one open discrepancy for `amount`, and the card shows both values.
- An offline bank yields `UNAVAILABLE` without breaking the page.
- A slow bank returns `PENDING`, then a real result on retry after the interval.
- Accept is refused while a discrepancy is open, and allowed after resolution with a note.
- Editing the amount after confirmation resets that field to `CLAIMED`.

Phase 4: Rules engine, alerts and escalation

Goal: the system watches dates and states, creates alerts, escalates what is ignored, and closes alerts that no longer apply. It must behave identically whether time passes for real or the clock is advanced by hand.

4.1 Schema (migration 004)

```
CREATE TABLE alerts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  instrument_id uuid NOT NULL REFERENCES instruments(id),
  kind text NOT NULL,              -- MATURITY_REMINDER |
PENDING_VERIFICATION | DISCREPANCY_OPEN | OVERDUE_RENEWAL | RELEASE_PENDING
```

```
  rule_key text NOT NULL,          -- e.g. 'maturity:30'
  level int NOT NULL DEFAULT 0,    -- escalation level
  recipient_role user_role NOT NULL,
  recipient_user_id uuid REFERENCES users(id),
  due_date date NOT NULL,
  message text NOT NULL,
  status text NOT NULL DEFAULT 'OPEN',   -- OPEN | ACKNOWLEDGED | ESCALATED |
RESOLVED
  resolution text,                 -- how it closed: SYSTEM_CONDITION_CLEARED
| MANUAL | SUPERSEDED
  dedupe_key text NOT NULL UNIQUE, -- instrument_id|rule_key|due_date|level
  created_at timestamptz NOT NULL,
  resolved_at timestamptz
);
CREATE INDEX alerts_open ON alerts (status, recipient_role);

CREATE TABLE outbox (          -- stand-in for email, visible to admins
  id bigserial PRIMARY KEY,
  to_role user_role, to_user_id uuid, to_institution_id uuid,
  subject text NOT NULL, body text NOT NULL,
  created_at timestamptz NOT NULL
);
```

The unique `dedupe_key` is what makes the job idempotent: inserting the same alert twice does nothing.

4.2 Rule configuration

Rules live in the `rules` table as JSON (shown here in JavaScript notation).

```
// type 'reminder'
{ offsets: [
    { days: 90, recipients: ['institution', 'officer'] },
    { days: 60, recipients: ['institution', 'officer'] },
    { days: 30, recipients: ['institution', 'officer'] },
    { days: 7,  recipients: ['institution', 'officer', 'approver'] } ] }
// type 'pending_verification'
{ afterDays: 5, recipients: ['officer'] }
// type 'discrepancy_open'
{ afterDays: 7, recipients: ['officer'] }
// type 'overdue_renewal'
{ graceDays: 15, recipients: ['institution', 'officer', 'approver'] }
// type 'release_pending'
{ afterDays: 5, recipients: ['approver'] }
// type 'escalation'
{ levels: [ { level: 1, role: 'officer',  afterDays: 0 },
            { level: 2, role: 'approver', afterDays: 7 },
            { level: 3, role: 'admin',    afterDays: 14 } ] }
```

Read the rules at the start of each run. Changing a rule in the admin screen (Phase 8) therefore takes effect on the next run with no deployment.

4.3 The evaluation job

`evaluate(asOf)` runs in one transaction, guarded by an advisory lock so two runs cannot overlap (`SELECT pg_advisory_xact_lock(42)`). It returns a summary object. Run its steps in this order.

Step 1: status sweeps, applied one after another so a large clock jump cascades correctly

1. `ACTIVE` to `EXPIRING` when `maturity_date - asOf <= the largest reminder offset` (90 days by default).
2. `EXPIRING` to `MATURED` when `maturity_date <= asOf`.
3. `MATURED` to `LAPSED` when `asOf > maturity_date + graceDays` and no renewal instrument exists for it.

Each change goes through `transition()` with the actor `system`. Instruments in `DRAFT`, `SUBMITTED`, `UNDER_SCRUTINY`, `RETURNED`, `RENEWED`, `RELEASE_REQUESTED` and `RELEASED` are not swept.

Step 2: maturity reminders

For each live instrument compute `daysLeft`. Find the tightest offset already crossed (`daysLeft <= offset`). Create one alert for that offset per recipient role. Insert every looser offset as an already-resolved alert with `resolution = 'SUPERSEDED'`, so a jump from day 100 to day 10 produces one alert, not four.

Step 3: process alerts

- Pending verification: instruments in `SUBMITTED` or `UNDER_SCRUTINY` with `verification_state` in (`UNVERIFIED`, `PENDING`, `UNAVAILABLE`) older than `afterDays`.
- Open discrepancy: any `OPEN` discrepancy older than `afterDays`.
- Overdue renewal: `MATURED` or `LAPSED` instruments with no successor.
- Release pending: release requests waiting longer than `afterDays`.

Step 4: escalation

For each `OPEN` or `ACKNOWLEDGED` alert, compute its age in days since creation. If the age reaches the next level's `afterDays`, insert the next-level alert (new `dedupe_key` because level is part of it), set the old alert to `ESCALATED`, and write an event.

Step 5: auto-resolution

Each alert kind has a function `stillRelevant(alert, state)`. Close alerts whose condition no longer holds (verification confirmed, discrepancy resolved, renewal accepted, instrument released) with `resolution = 'SYSTEM_CONDITION_CLEARED'`.

```
async function evaluate(tx, asOf: string) {
  await tx.none('SELECT pg_advisory_xact_lock(42)');
  const rules = await loadRules(tx);
  const s = { statusChanges: 0, alertsCreated: 0, escalations: 0, resolved: 0
};
  s.statusChanges += await sweepStatuses(tx, asOf, rules);
  s.alertsCreated += await maturityReminders(tx, asOf, rules);
  s.alertsCreated += await processAlerts(tx, asOf, rules);
  s.escalations   += await escalate(tx, asOf, rules);
  s.resolved      += await autoResolve(tx, asOf);
  return s;
}
```

Every insert uses `ON CONFLICT (dedupe_key) DO NOTHING`, so a second run on the same date changes nothing and returns zeros.

4.4 Recipients and the outbox

- An alert addressed to `officer` goes to the instrument's `assigned_officer_id` if one exists, otherwise to the officer pool.
- An alert addressed to `institution` is visible only to users of that instrument's institution.
- For each new alert, write an `outbox` row (subject, body, target). A page for admins lists the outbox, which stands in for email in a demo. Real email can replace the insert later without changing anything else.

4.5 Scheduling

- Real clock: run `evaluate` at start-up and daily at 06:00 IST with a scheduler such as node-cron.
- Simulated clock: the `POST /clock/advance` and `/clock/set` endpoints call `evaluate` immediately after changing the clock, in the same request, and return the summary.
- `POST /jobs/evaluate` (admin) runs it on demand.

4.6 Alert endpoints

Method and path	Purpose
`GET /alerts`	Filter by status, kind, instrument; scoped by role and institution
`GET /alerts/summary`	Open count per kind, for the bell badge
`POST /alerts/:id/acknowledge`	Mark as seen; does not stop escalation
`POST /alerts/:id/resolve`	Manual close with a note

Acknowledging does not stop escalation on purpose: only fixing the cause, or a recorded manual resolution, does.

4.7 Frontend

- **Clock widget** in the header for admins: a persistent badge `SIMULATED: 12 Dec 2026` when not in real mode, with buttons +1 day, +7, +30, Set date and Reset. After each change show a toast built from the summary, for example `3 status changes, 14 alerts, 2 escalations`.

- **Alerts inbox:** grouped by instrument, filterable, with kind chips, level indicators and a link to the instrument.
- **Bell** with the open count from `/alerts/summary`.
- **Outbox page** for admins.
- Instrument tables show a chip for the worst open alert level.

4.8 Edge cases

- Moving the clock backwards is allowed but warns the user. It never deletes alerts, and the job only creates and closes.
- Instruments whose maturity falls on a weekend or holiday still use calendar days. Business-day logic is out of scope.
- A renewal accepted during the same run as a sweep must win: run auto-resolution last, and make the `LAPSED` sweep check for a successor.
- Large jumps (a year) must finish well within the request timeout. Keep sweeps set-based with indexes on `status` and `maturity_date`.

4.9 Checks

- Advance 45 days: instruments inside 90 days become `EXPIRING`, the correct reminder alerts exist, and counts match expectations.
- Running `evaluate` twice on one date creates nothing the second time.
- Jump from day 100 to day 10 before maturity creates one reminder alert and superseded older ones.
- An unacknowledged alert escalates one level after the configured days, and the original becomes `ESCALATED`.
- Confirming a verification closes the matching pending-verification alert.

Phase 5: Renewal chain and instrument passport

Goal: renewed or replacement instruments link to what they replace, and each instrument has a single page that tells its whole story.

5.1 Schema changes (migration 005)

```
ALTER TABLE instruments ADD CONSTRAINT no_self_parent CHECK
(parent_instrument_id <> id);
CREATE UNIQUE INDEX one_successor_per_instrument ON instruments
(parent_instrument_id)
  WHERE parent_instrument_id IS NOT NULL;
ALTER TABLE observations ADD COLUMN visible_to_institution boolean NOT NULL
DEFAULT false;
```

The unique index means an instrument can have at most one successor. Drafts can be deleted, so an abandoned replacement does not block a new one.

5.2 Starting a renewal

`POST /instruments/:id/renew` (institution role).

- Allowed when the parent is `EXPIRING`, `MATURED` or `LAPSED` and has no successor.
- Creates a new `DRAFT` instrument with `parent_instrument_id` set, copying the institution, type, bank and beneficiary name. Amount defaults to the parent's amount. Dates are left empty.
- Returns the new instrument, which then goes through the normal Phase 2 submission and Phase 3 scrutiny.
- Writes an event on both instruments (`RENEWAL_STARTED`).

5.3 Rules for a replacement

Checked at submit time and again at accept time. Make each configurable in the rules table.

- `maturity_date` must be later than the parent's `maturity_date`, otherwise `422`.
- If `issue_date` is later than the parent's `maturity_date`, there is a coverage gap. Record an observation of severity `CONCERN` stating the number of days.
- If the amount is lower than the parent's, record a `CONCERN` observation naming both amounts. Officers decide whether it is acceptable.
- Lien marking must be confirmed on the replacement through the same document check as other fields.

5.4 Activating the chain

Inside the accept service (Phase 3), after the replacement moves to `ACTIVE`:

1. If `parent_instrument_id` is set, call `transition(parent, 'RENEWED')` with the actor and `extra = { renewedBy: child.id }`.
2. Write `RENEWED_BY` on the parent and `RENEWS` on the child.
3. Let the auto-resolution step (Phase 4) close the parent's reminders and overdue alerts on the next run. Also trigger a targeted resolve right away so the screen is correct immediately.

The parent is never edited otherwise. It keeps its original values forever.

5.5 Chain query

```
WITH RECURSIVE up AS (
  SELECT * FROM instruments WHERE id = $1
  UNION ALL
  SELECT p.* FROM instruments p JOIN up ON up.parent_instrument_id = p.id
), down AS (
  SELECT * FROM instruments WHERE parent_instrument_id = $1
  UNION ALL
  SELECT c.* FROM instruments c JOIN down ON c.parent_instrument_id = down.id
)
SELECT * FROM up UNION SELECT * FROM down ORDER BY issue_date;
```

Compute gaps and overlaps between consecutive links in the service: the number of days between one link's maturity and the next link's issue date (negative means overlap).

5.6 The passport endpoint

`GET /instruments/:id/passport` returns one document so the page needs a single request:

- Instrument and the six fields with their trust tiers and sources
- Chain with gap or overlap annotations
- Documents (metadata only), verifications, discrepancies, observations, alerts, release requests
- Computed summary: days to maturity, weakest tier, open problems from `findGaps`, whether an override acceptance happened
- Timeline: events for this instrument and for the entities attached to it, ordered by time

Build the timeline from the event log with a label map, for example `STATUS_ACTIVE` becomes *Accepted by officer* and `VERIFICATION_RESULT` becomes *Bank verification: mismatch on amount*. Keep this map in one file so labels stay consistent.

Visibility by role. Institution users see the instrument, their documents, verification outcomes, observations marked `visible_to_institution`, public alerts and a whitelisted subset of events. Internal officer observations and escalation details stay hidden. Officers, approvers, admins and auditors see everything.

5.7 Passport page

- **Header:** reference, bank, amount, status chip, verification chip, the weakest trust tier, days to maturity.
- **Tabs:** Timeline (default), Fields, Documents, Verification, Chain, Alerts, Release.
- **Timeline:** vertical list with icon, label, actor, time on the app clock and an expandable before and after view of the change.
- **Chain component:** horizontal nodes joined by arrows. Each node shows reference, bank, amount, dates and a status chip, the current instrument is highlighted, and each arrow carries the gap or overlap in days.
- **Fields tab:** each field with its trust badge, source and last-updated time.

5.8 Checks

- Start a renewal from an expiring instrument, complete it, and see the parent become `RENEWED`, the child `ACTIVE`, and the chain show both with the correct gap.
- A replacement with an earlier maturity is rejected.
- A second renewal on the same parent is refused.
- An institution user does not see internal observations on the passport.
- Parent alerts close after the child is accepted.

Phase 6: Release, discharge and refund workflow

Goal: release requests pass through conditions and approvals, and cannot get past an open problem.

6.1 Schema (migration 006)

```
CREATE TABLE release_requests (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  instrument_id uuid NOT NULL REFERENCES instruments(id),
  requested_by uuid NOT NULL REFERENCES users(id),
  reason text NOT NULL,
  status text NOT NULL DEFAULT 'PENDING_CONDITIONS',
    -- PENDING_CONDITIONS | PENDING_APPROVAL | APPROVED | REJECTED | RETURNED
| CANCELLED
  refund_reference text,
  requested_at timestamptz NOT NULL,
  closed_at timestamptz
);
CREATE UNIQUE INDEX one_open_release_per_instrument ON release_requests
(instrument_id)
  WHERE status IN ('PENDING_CONDITIONS','PENDING_APPROVAL','RETURNED');

CREATE TABLE release_conditions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  release_request_id uuid NOT NULL REFERENCES release_requests(id) ON DELETE
CASCADE,
  code text NOT NULL,
  description text NOT NULL,
  mandatory boolean NOT NULL DEFAULT true,
  auto boolean NOT NULL DEFAULT false,
  met boolean NOT NULL DEFAULT false,      -- for manual conditions
  checked_by uuid REFERENCES users(id),
  checked_at timestamptz,
  remarks text
);

CREATE TABLE release_approvals (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  release_request_id uuid NOT NULL REFERENCES release_requests(id) ON DELETE
CASCADE,
  level int NOT NULL,
  role user_role NOT NULL,
  approver_id uuid NOT NULL REFERENCES users(id),
  decision text NOT NULL,       -- RECOMMEND | APPROVE | REJECT | RETURN
  remarks text NOT NULL,
  decided_at timestamptz NOT NULL
```

```
);

ALTER TABLE documents ADD COLUMN release_request_id uuid REFERENCES
release_requests(id);
```

6.2 Conditions

Define them in a `release_conditions` rule so they are configurable. Defaults:

Code	Type	Meaning
`NO_OPEN_DISCREPANCY`	auto	No open discrepancy on the instrument
`NO_BLOCKING_OBSERVATION`	auto	No unresolved observation of severity `BLOCKING`
`INSTRUMENT_VERIFIED`	auto	`verification_state` is `CONFIRMED`, or an override acceptance is on record
`REQUEST_DOCUMENTS_ATTACHED`	auto	At least one release document is attached
`COMPLIANCE_CONFIRMED`	manual	The requirement the deposit secured is confirmed as fulfilled
`LIEN_REMOVAL_DOCUMENT`	manual	Bank letter or request for lien removal is on file

Auto conditions are evaluated live every time they are read, never stored as met. This is what lets the system block a decision when something changes after a recommendation. Manual conditions are ticked by an officer with remarks and stored. When a request is created, copy the configured conditions into `release_conditions`.

6.3 Flow

1. **Request.** `POST /instruments/:id/release-requests {reason}` (institution). Allowed when the instrument is `ACTIVE`, `EXPIRING`, `MATURED` or `LAPSED` with no open request. Store the current status in `status_before_release`, `transition` to `RELEASE_REQUESTED`, create the request in `PENDING_CONDITIONS` and copy the conditions. Release documents are uploaded with `release_request_id` set.
2. **Conditions.** `PATCH /release-conditions/:id {met, remarks}` (officer) for manual items. `GET /release-requests/:id` returns every condition with its live state and, for failing auto conditions, a reason and a pointer to the cause.

3. **Recommend.** `POST /release-requests/:id/recommend {remarks}` (officer). Refused with `409` and the reasons unless every mandatory condition is met. Records a `RECOMMEND` row at level 1 and moves the request to `PENDING_APPROVAL`.
4. **Decide.** `POST /release-requests/:id/decide {decision, remarks}` (approver). Remarks are mandatory for every decision.
   - Recompute all auto conditions first. If any mandatory condition fails, return `409` with reasons. This is the block.
   - **APPROVE:** `transition(instrument, 'RELEASED')`, set `closed_at`, store `refund_reference` if given, and write an event whose `after_json` is a **snapshot of every condition and its state at decision time**.
   - **REJECT:** restore `status_before_release`, set the request `REJECTED`.
   - **RETURN:** set the request `RETURNED`. The institution can add information and resubmit, which sets it back to `PENDING_CONDITIONS`. The instrument stays `RELEASE_REQUESTED`.

6.4 Controls

- **Separation of duties:** the approver must not be the same user who recommended.
- **Concurrency:** lock the request and instrument rows with `FOR UPDATE` in every step so two decisions cannot race.
- **Configurable levels:** a rule such as `{ levels: [{ level: 1, role: 'officer', action: 'RECOMMEND' }, { level: 2, role: 'approver', action: 'DECIDE' }], extraApprovalAbove: 5000000 }` adds a further approver for large amounts. Implement the second level generally (a loop over configured levels) rather than special-casing it.
- **Human decision:** nothing in the system approves a release automatically. The screen says so.

6.5 Interaction with other phases

- Maturity reminders for an instrument in `RELEASE_REQUESTED` stay visible, but the evaluation job adds a `RELEASE_PENDING` alert when the request waits too long.
- On `RELEASED`, auto-resolution closes every open alert on the instrument.
- A `RENEWED` instrument cannot be released (terminal). Only the live successor can.

6.6 Frontend

- **Institution:** a *Request release* button on eligible instruments opens a form with a reason and document upload. The instrument page then shows request status and any return message.
- **Officer:** a release review page with the conditions checklist. Auto conditions show live green or red with a link to the cause (for example *Open discrepancy on amount*, linking to the discrepancy card). Manual conditions are checkboxes with a remarks field. The Recommend button is disabled while mandatory conditions fail, and lists why.
- **Approver:** a decision page with the summary, the passport link, the condition snapshot, and Approve, Return and Reject buttons with mandatory remarks. A banner states that the portal supports the decision and the decision rests with the competent authority.
- **Blocked state:** a clear banner listing the failing conditions, shown to everyone with access, not only the approver.

6.7 Edge cases

- A discrepancy opened after the recommendation blocks the decision, and the approver sees why.
- An instrument that matures or lapses while the request is open keeps its request. The restored status on rejection is recomputed by the next evaluation run if time has passed.
- Two approvers clicking at once: the second gets a clear already-decided message.
- A request returned and resubmitted keeps its history in the approvals table.

6.8 Checks

- A request cannot be recommended while an auto condition fails, and the response names it.
- Open a discrepancy after recommendation: the approver's decision is refused with the reason.
- After approval the instrument is `RELEASED`, alerts are closed, and the event contains the condition snapshot.
- Rejection restores the earlier status.
- The recommending officer cannot also approve.